

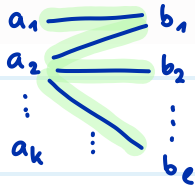
Quiz

$\Leftrightarrow$ bipartit

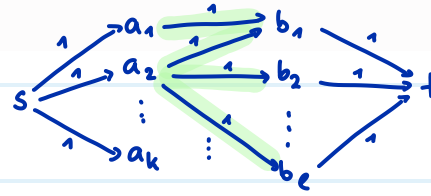
Für jeden 2-färbbaren Graph G können wir mithilfe eines Flussproblems herausfinden, ob G ein perfektes Matching hat.

☒ Wahr
☐ Falsch

Sei $G = (A \cup B, E)$



perfektes Matching $\Leftrightarrow \max \text{flow} = |A| = |B|$

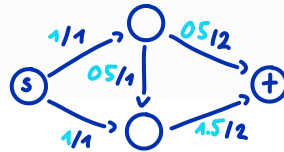


Sei $N = (V, A, c, s, t)$ ein Netzwerk. Wenn c nur ganzzahlige Kantengewichte hat, so ist jeder maximale Fluss in N ganzzahlig.

Kapazitäten

☐ Wahr
☒ Falsch

Gegenbeispiel:



Fluss f ist ganzzahlig $\Leftrightarrow \forall e \in E: f(e) \in \mathbb{Z}$

Sei $N = (V, A, c, s, t)$ ein Netzwerk ohne entgegen gerichtete Kanten und f ein Fluss mit $val(f) > 0$. Sei N_f das Restnetzwerk. Dann enthält N_f einen gerichteten Weg von t nach s .

☒ Wahr
☐ Falsch

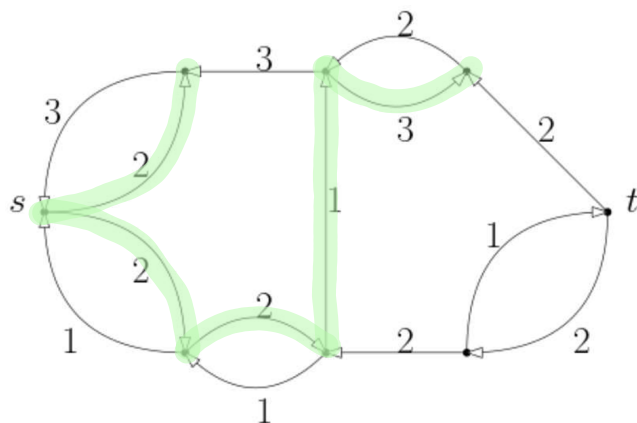
weil $val(f) > 0$, gibt es einen "Tropfen" der von s nach t fließt

Das könnten wir rückgängig machen Dieser umgekehrte Weg des Tropfens ist ein gerichteter Weg von t nach s in N_f

Sei $N = (V, A, c, s, t)$ ein Netzwerk ohne entgegen gerichtete Kanten und f ein Fluss. Das Restnetzwerk N_f ist gegeben (siehe Graph-Szenario). Ist der Fluss maximal?

☒ Wahr
☐ Falsch

weil $\nexists s-t$ Pfad



erreichbar von s

Sei $N = (V, A, c, s, t)$ ein Netzwerk ohne entgegen gerichtete Kanten und sei die Kapazität jeder Kante höchstens U . Dann berechnet der Ford-Fulkerson Algorithmus in Zeit $O(mnU)$ einen maximalen Fluss.

(gemeine Frage)

☐ Wahr
☒ Falsch

$O(mnU)$ wurde nur für ganzzahlige Kapazitäten gezeigt

Sei $N = (V, A, c, s, t)$ ein Netzwerk, f ein Fluss in N und (S, T) ein s - t -Schnitt in N . Dann gilt $val(f) \geq cap(S, T)$.

☐ Wahr
☒ Falsch

anders herum $val(f) \leq cap(S, T)$

Sei f ein maximaler Fluss in einem Netzwerk $N = (V, A, c, s, t)$. Dann gibt es keine zwei Knoten u, v in V , sodass sowohl die Kante (u, v) als auch die Kante (v, u) im Restnetzwerk N_f vorkommt.

☐ Wahr
☒ Falsch

Gegenbeispiel

N :



N_f :



Das MIN-CUT-Problem kann gelöst werden, indem man einen Knoten s fest wählt und minimale s - t -Schnitte für alle $t \in V \setminus \{s\}$ berechnet.

☒ Wahr
☐ Falsch

Wenn e eine Kante von G ist, dann gilt immer $\mu(G/e) \leq \mu(G)$.

☐ Wahr
☒ Falsch

anders herum: $\mu(G/e) \geq \mu(G)$

Wenn es einen minimalen Schnitt C von G mit $e \notin C$ gibt, dann gilt $\mu(G/e) = \mu(G)$.

☒ Wahr
☐ Falsch

Lemma Skript

Min-Cut Problem

Multigraph: $G = (V, E)$ ohne Schleifen, ungerichtet, ungewichtet, mehrere Kanten zwischen zwei Knoten sind erlaubt

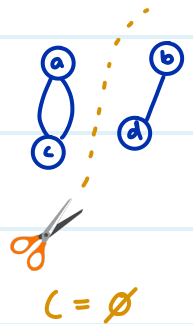
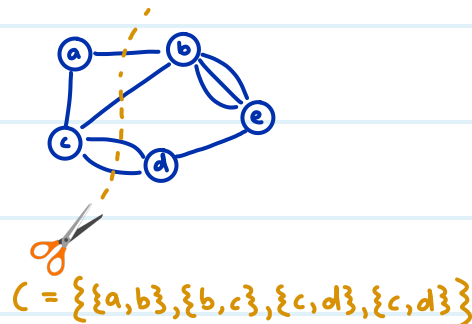
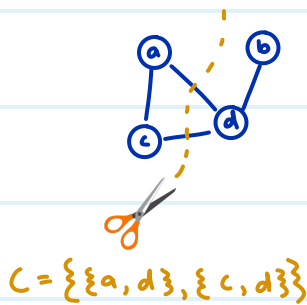
Der Grad ist die Anzahl anliegender Kanten, und nicht die Anzahl Nachbarn



$$\deg(a) = 3, \deg(b) = 5, \deg(c) = 2$$

← "C" für Cut

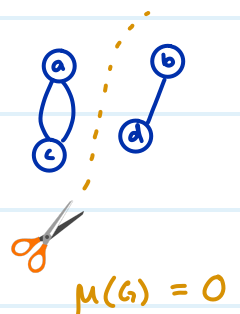
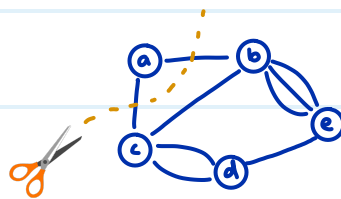
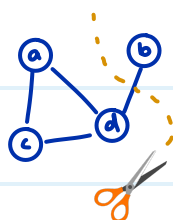
Kantenschnitt Eine Teilmenge der Kanten $C \subseteq E$, sodass $(V, E \setminus C)$ unzusammenhängend ist (= edge cut)



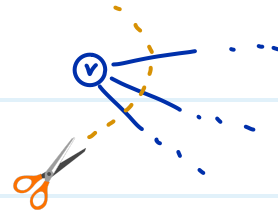
Achtung: dieser Cut ist nicht dasselbe wie der Cut bei Flüssen

← "μ" für min

$\mu(G)$: Kardinalität $|C|$ des kleinstmöglichen Schnitts C im Multigraphen G



Es gilt immer: $\mu(G) \leq \min_{v \in V} \deg(v)$
kleinster Grad



Wir könnten einfach diesen Knoten herausschneiden

Min-Cut Algorithmus via Flüsse

1) Gegeben einen Multigraphen G , konstruieren wir ein Netzwerk $N = (V, A, c, s, t)$

wobei • N dieselben Knoten hat wie G

• A hat eine Kante (u, v) und (v, u) falls G die Kante $\{u, v\}$ hat

• $c(u, v) = \text{Anzahl Kanten zwischen } u \text{ und } v \text{ in } G$

• s ist ein beliebiger, fixer Knoten

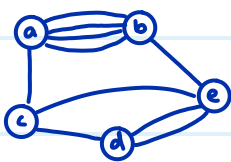
• $t \dots$ (nächster Schritt)

2) Wir iterieren über alle möglichen targets $t \in V \setminus \{s\}$ und berechnen die Kapazität

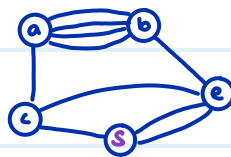
$\text{cap}(S, T)$ des minimalen Schnitts (S, T) mit dem maxflow Algorithmus

3) Gib die kleinste Kapazität aus, die gefunden wurde

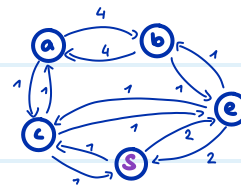
Bsp 1) G :



wir wählen $s = d$

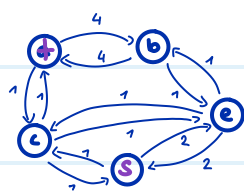


N :

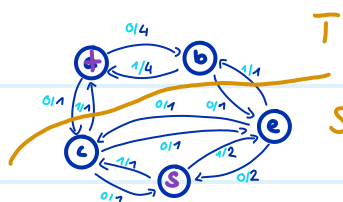


2) Wir iterieren über alle targets $t \in \{a, b, c, e\}$:

$t = a$:

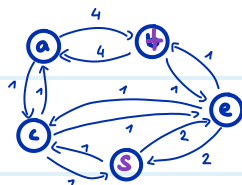


max flow Algorithmus

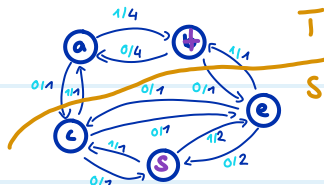


$$\max \text{flow} = \min \text{cap}(S, T) = 2$$

$t = b$

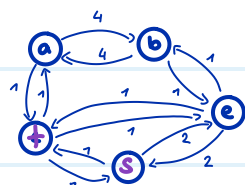


max flow Algorithmus

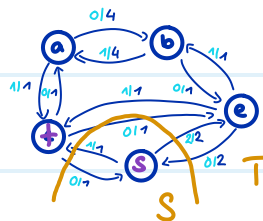


$$\max \text{flow} = \min \text{cap}(S, T) = 2$$

$t = c$:

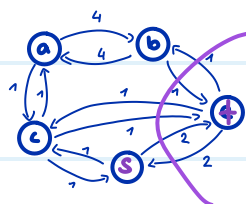


max flow Algorithmus

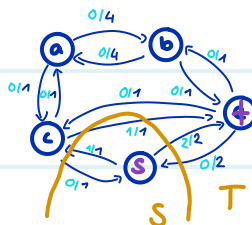


$$\max \text{flow} = \min \text{cap}(S, T) = 3$$

$t = e$:



max flow Algorithmus



$$\max \text{flow} = \min \text{cap}(S, T) = 3$$

$$3.) \mu(G) = \min \{2, 2, 3, 3\} = \underline{\underline{2}}$$

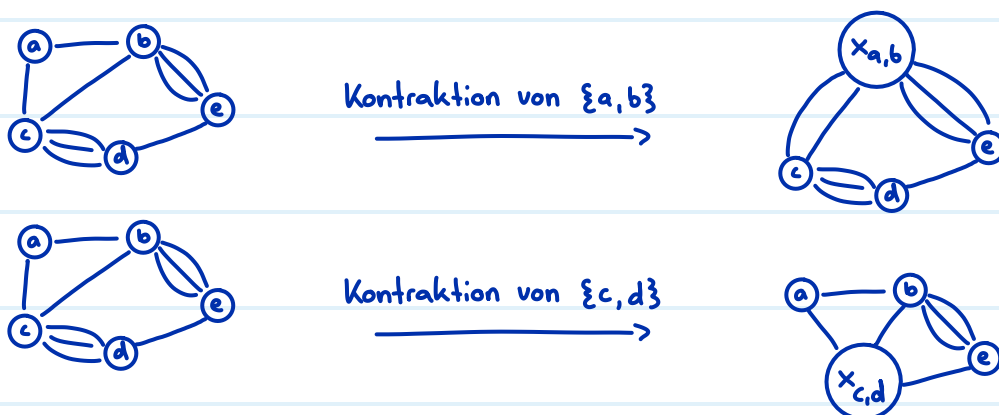
Intuition: $\text{Cap}(S, T)$ ist genau die minimale Anzahl an Kanten, die man entfernen muss, damit s und t nicht mehr verbunden sind in G

Laufzeit: $O(n \cdot \underbrace{n^3 \log n}_{\substack{\text{über } t \\ \text{iterieren}}} = O(n^4 \cdot \log n)$
 max flow in $O(m n \log n)$ mit $m \leq n^2$

Kontraktionen

Kontraktion einer Kante $e = \{u, v\}$:

- u und v werden zu einem Knoten $x_{u,v}$
- Kanten zwischen u und v werden entfernt
- andere Kanten nach u oder v gehen nun zu $x_{u,v}$
- den entstehenden Graphen nennen wir G/e

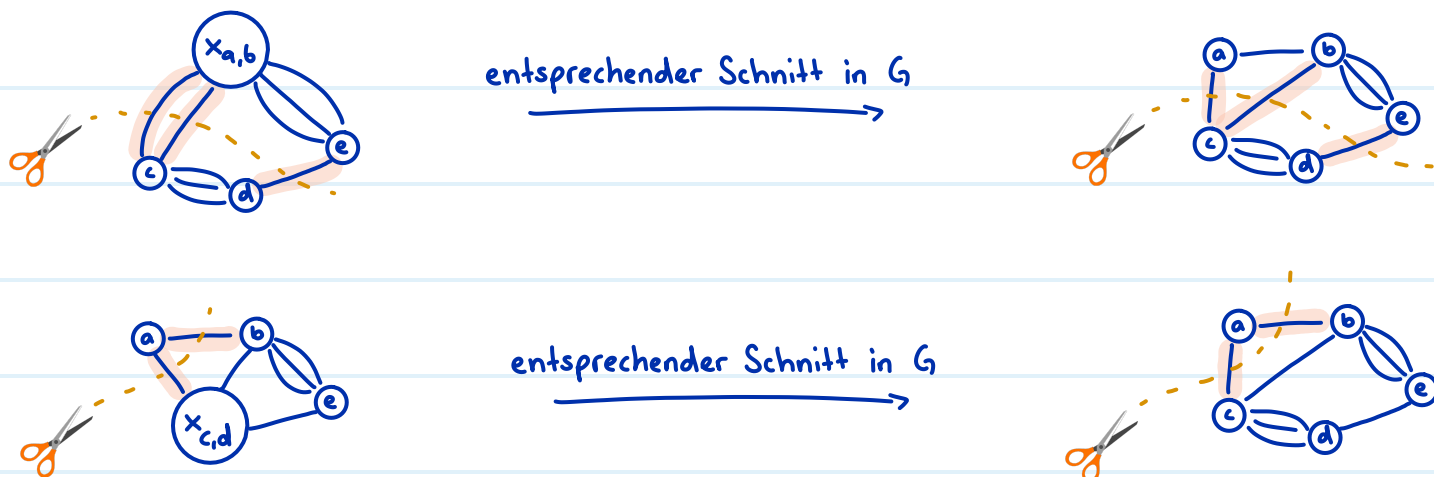


Für alle Kanten e gilt: $\mu(G/e) \geq \mu(G)$

Das heisst, durch Kontraktionen wird der minimale Kantenschnitt grösser oder er bleibt gleich.

Beweis: Sei C ein minimaler Schnitt in G/e

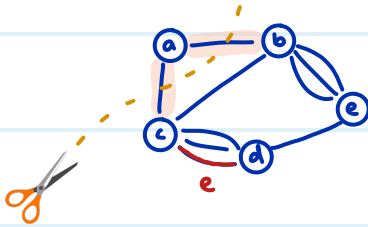
Dann gibt es auch einen Schnitt in G mit "denselben" Kanten



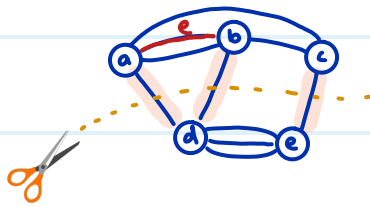
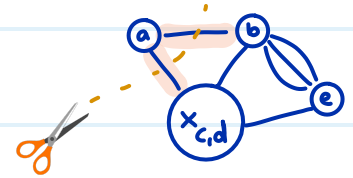
Falls G einen minimalen Kantenschnitt C mit $e \notin C$ hat, dann ist $\mu(G/e) = \mu(G)$

Beweis: Sei C ein minimaler Schnitt in G mit $e \notin C$

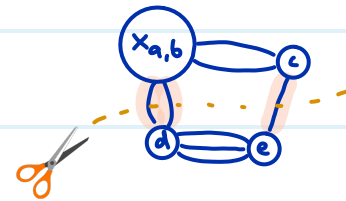
Dann gibt es auch einen Schnitt in G/e mit "denselben" Kanten



entsprechender Schnitt in G/e



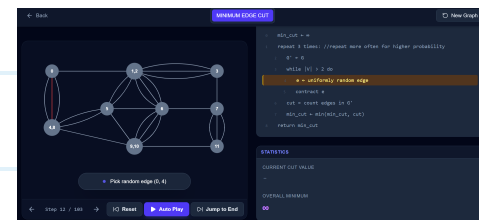
entsprechender Schnitt in G/e



Algorithmus:

$\text{CUT}(G)$ in $O(n^2)$ G zusammenhängender Multigraph

- 1: $G' \leftarrow G$
- 2: **while** $|V(G')| > 2$ **do** $\rightarrow O(n)$ Iterationen
- 3: $e \leftarrow$ gleichverteilt zufällige Kante in G'
- 4: $G' \leftarrow G'/e \rightarrow$ in $O(n)$
- 5: **return** Grösse des eindeutigen Schnitts in G'



• Monte-Carlo Algorithmus

Falls e zufällig gleichverteilt ausgewählt wird, gilt $\Pr[\mu(G) = \mu(G/e)] \geq 1 - \frac{n}{2}$

Beweis: Sei C ein minimaler Kantenschnitt und $e \in E$ zufällig gleichverteilt

$$|E| = \frac{1}{2} \sum_{v \in V} \deg(v)$$

Handschlag Lemma

$$\Rightarrow |E| \geq \frac{1}{2} \sum_{v \in V} |C|$$

weil für alle $v \in V$ gilt: $\deg(v) \geq \underbrace{\mu(G)}_{=|C|}$

$$\Rightarrow |E| \geq \frac{1}{2} n |C| \quad |V| = n$$

$$\begin{aligned} \Pr[\mu(G) = \mu(G/e)] &\geq \Pr[e \notin C] \\ &= 1 - \Pr[e \in C] \\ &= 1 - \frac{|C|}{|E|} \\ &\geq 1 - \frac{|C|}{\frac{1}{2} n |C|} \\ &= \underline{\underline{1 - \frac{2}{n}}} \end{aligned}$$

schon gezeigt

Gegenwahrscheinlichkeit

zufällig gleichverteilt $\rightarrow$ Laplace Raum

$$|E| \geq \frac{1}{2} n |C|$$

kürzen

Wir definieren: $\hat{p}(G) := \Pr[\text{Cut}(G) \text{ ist korrekt}]$

(Erfolgswahrscheinlichkeit für G)

$$\text{und } \hat{p}(n) := \inf_{G=(V,E), |V|=n} \hat{p}(G)$$

(Erfolgswahrscheinlichkeit für den schwierigsten Graphen mit n Knoten)

$$\text{Es gilt } \forall n \geq 3: \hat{p}(n) \geq \frac{2}{n(n-1)}$$

Beweis Sei G ein Multigraph mit n Knoten

"Cut(G) ist korrekt" tritt genau dann ein, wenn folgende beide Ereignisse eintreten

$$E_1 := \mu(G) = \mu(G/e) \quad (= \text{erste Kontraktion erhält den min cut})$$

$$E_2 := \text{Cut}(G/e) \text{ ist korrekt} \quad (= \text{restliche Kontraktionen erhalten den min cut})$$

$$\text{Das heisst } \hat{p}(G) = \Pr[E_1 \wedge E_2]$$

$$= \Pr[E_1] \Pr[E_2 | E_1] \quad (\text{Multiplikationssatz})$$

$$\geq \left(1 - \frac{2}{n}\right) \hat{p}(n-1)$$

($\Pr[E_1] \geq 1 - \frac{2}{n}$ wurde oben gezeigt,
und $\Pr[E_2 | E_1] \geq \hat{p}(n-1)$ per Definition)

Weil dies für jeden beliebigen Multigraphen mit n Knoten gilt, folgt:

$$\hat{p}(n) \geq \left(1 - \frac{2}{n}\right) \hat{p}(n-1)$$

Durch wiederholtes Einsetzen ergibt sich:

$$\begin{aligned} \hat{p}(n) &\geq \frac{n-2}{n} \cdot \hat{p}(n-1) && \left(1 - \frac{2}{n} = \frac{n-2}{n}\right) \\ &\geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \hat{p}(n-2) \\ &\vdots \\ &\geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \frac{n-5}{n-3} \cdot \dots \cdot \frac{4}{6} \cdot \frac{3}{5} \cdot \frac{2}{4} \cdot \frac{1}{3} \cdot \hat{p}(2) \\ &= \frac{\cancel{n-2}}{n} \cdot \frac{\cancel{n-3}}{\cancel{n-1}} \cdot \frac{\cancel{n-4}}{\cancel{n-2}} \cdot \frac{\cancel{n-5}}{\cancel{n-3}} \cdot \dots \cdot \frac{\cancel{4}}{\cancel{6}} \cdot \frac{\cancel{3}}{\cancel{5}} \cdot \frac{\cancel{2}}{\cancel{4}} \cdot \frac{1}{3} \cdot \underbrace{\hat{p}(2)}_{=1, \text{ trivialer Fall}} \\ &= \frac{2}{n(n-1)} \end{aligned}$$

Fehlerreduktion für $\text{Cut}(G)$

• aktuelle Korrektheit: $\varepsilon \geq \frac{2}{n(n-1)}$

• Zielfehler: $\delta = e^{-\lambda}$

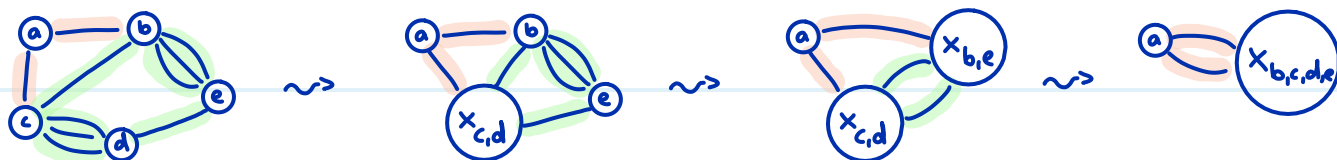
$$\Rightarrow N = \lceil \varepsilon^{-1} \ln(\delta^{-1}) \rceil = \lceil \frac{n(n-1)}{2} \ln(e^\lambda) \rceil = \lceil \lambda \frac{n(n-1)}{2} \rceil \text{ Wiederholungen, von denen wir}$$

am Schluss den kleinst gefundenen Schnitt ausgeben

Laufzeit in $O(\underbrace{n^2}_{\text{Laufzeit von Cut}(G)} \cdot \underbrace{\lambda n^2}_{\text{Wiederholungen}}) = O(\lambda n^4) \rightarrow \text{schlecht}$

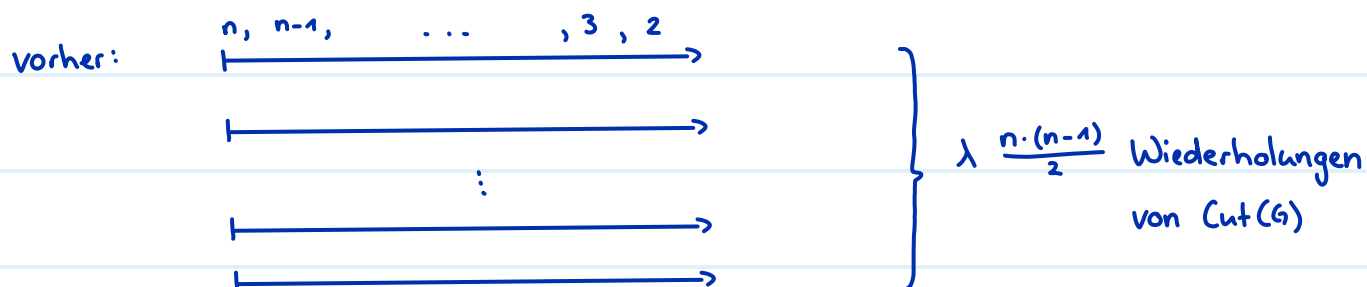
Bootstrapping

Feststellung: umso kleiner der Graph wird, desto wahrscheinlicher ist es, dass der Algorithmus einen Fehler macht

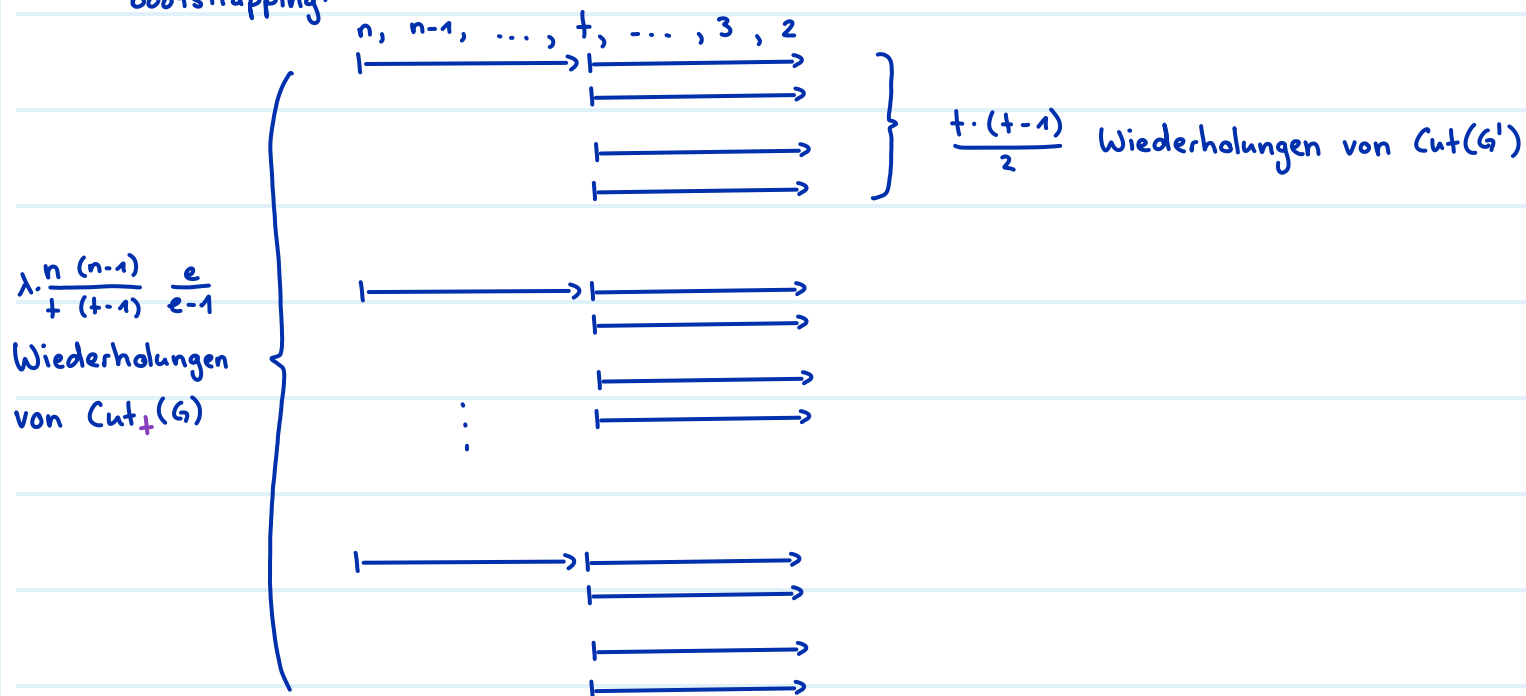


rote Kante kontrahieren = Fehler, grüne Kante kontrahieren = min cut bleibt erhalten

Idee: vorsichtiger sein wo es gefährlich ist



bootstrapping:



$\text{CUT}_t(G)$ G zusammenhängender Multigraph

- 1: $G' \leftarrow G$
- 2: **while** $|V(G')| > t$ do $\rightarrow$ bei t übrigen Knoten abbrechen
- 3: $e \leftarrow$ gleichverteilt zufällige Kante in G'
- 4: $G' \leftarrow G'/e$
- 5: **return** Resultat des $O(t^4)$ Algorithmus auf G'

$= \lambda \frac{t(t-1)}{2}$ Wiederholungen von $\text{Cut}(G')$

mit $\lambda=1 \Rightarrow$ Erfolgswahrscheinlichkeit $\geq 1 - e^{-\lambda} = 1 - e^{-1} = \frac{e-1}{e}$

Wir definieren: $\hat{p}_+(G) := \Pr[\text{Cut}_+(G) \text{ ist korrekt}]$ (Erfolgswahrscheinlichkeit für G)

und $\hat{p}_+(n) := \inf_{G=(V,E), |V|=n} \hat{p}_+(G)$

(Erfolgswahrscheinlichkeit für den schwierigsten Graphen mit n Knoten)

Analog erhalten wir

$$\hat{p}_+(n) \geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \dots \cdot \frac{t}{t+2} \cdot \frac{t-1}{t+1} \underbrace{\hat{p}_+(t)}_{\geq \frac{e-1}{e}}$$

$$\geq \frac{n-2}{n} \cdot \frac{n-3}{n-1} \cdot \frac{n-4}{n-2} \cdot \dots \cdot \frac{t}{t+2} \cdot \frac{t-1}{t+1} \cdot \frac{e-1}{e}$$

$$= \frac{t(t-1)}{n(n-1)} \cdot \frac{e-1}{e}$$

Fehlerreduktion für $\text{Cut}_+(G)$

• aktuelle Korrektheit: $\epsilon \geq \frac{t(t-1)}{n(n-1)} \cdot \frac{e-1}{e}$

• Zielfehler: $\delta = e^{-\lambda}$

$$\Rightarrow N = \lceil \epsilon^{-1} \ln(\delta^{-1}) \rceil = \lceil \lambda \frac{n(n-1)}{t(t-1)} \cdot \frac{e}{e-1} \rceil \text{ Wiederholungen}$$

$$\text{Laufzeit in } O\left(\underbrace{\left(n \cdot \underbrace{(n-t)}_{\substack{\text{Anzahl Kontraktionen} \\ \text{bis } t \text{ Knoten übrig} \\ \text{bleiben}}}\right)}_{\substack{\text{Zeit für eine} \\ \text{Kontraktion}}} + t^4\right) \cdot \underbrace{\lambda \frac{n(n-1)}{t(t-1)} \cdot \frac{e}{e-1}}_{\substack{\frac{t(t-1)}{2} \text{ Wiederholungen} \\ \text{von } \text{Cut}(G') \text{ in jeweils } O(t^2)}} \leq O\left(\lambda \left(\frac{n^4}{t^2} + n^2 \cdot t^2\right)\right)$$

Wiederholungen von $\text{Cut}_+(G)$

Wie t wählen? $\rightarrow t$ ist optimal falls $\frac{n^4}{t^2} = n^2 \cdot t^2$

$$\Leftrightarrow n^4 = n^2 \cdot t^4$$

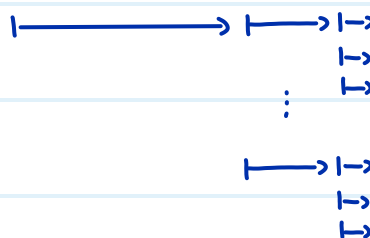
$$\Leftrightarrow n^2 = t^4$$

$$\Leftrightarrow t = \sqrt{n}$$

ergibt Laufzeit $O\left(\lambda \left(\frac{n^4}{n^2} + n^2 \sqrt{n^2}\right)\right) = O(\lambda n^3)$

Bootstrapping können wir beliebig oft machen, und erhalten im Limit die Laufzeit

$$O(n^2 \cdot \text{poly}(\log n))$$

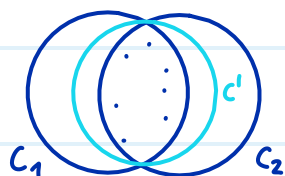


Smallest enclosing disk

Gegeben: eine endliche Punktmenge $P \subseteq \mathbb{R}^2$

Gesucht: der kleinste Kreis, der P umschließt. (Punkte auf dem Rand sind erlaubt)

Eindeutigkeit: Wir nehmen per Widerspruch an, dass $C_1 \neq C_2$ zwei kleinste umschließende Kreise sind. Dann können wir einen noch kleineren Kreis C' bestimmen, was ein Widerspruch zur Annahme ist, dass C_1 und C_2 kleinstmöglich sind.



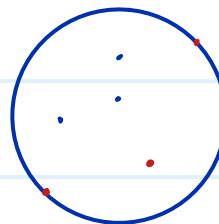
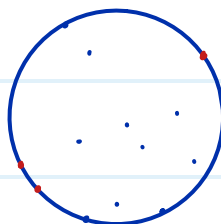
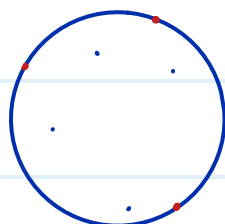
Lemma 3.26: Für jede Punktmenge P mit $|P| \geq 3$ gibt es eine Teilmenge $Q \subseteq P$,

sodass $|Q| = 3$ und $C(P) = C(Q)$

kleinster Kreis von P
= kleinster Kreis von Q

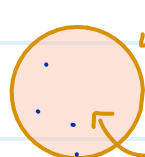
Q ist ein Zertifikat für die Minimalität des kleinsten Kreises

Q = rote Punkte



$\hookrightarrow Q$ muss nicht eindeutig sein

$C(P)$ vs $C^\circ(P)$



$C(P)$ = Randlinie

$C^\circ(P)$ = gesamte Fläche inklusive Randlinie

Brute Force Algorithmus:

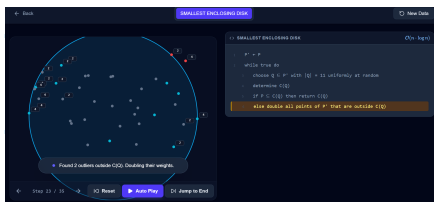
COMPLETE ENUMERATION(P) in $O(n^4)$

- 1: for all $Q \subseteq P$ mit $|Q| = 3$ do $\binom{n}{3} = \frac{n(n-1)(n-2)}{3 \cdot 2} \approx O(n^3)$ viele Möglichkeiten
- 2: bestimme $C(Q)$ in $O(1)$
- 3: if $P \subseteq C^\bullet(Q)$ then in $O(n)$, prüfen ob alle n Punkte aus P in der Kreisscheibe $C^\bullet(Q)$ enthalten sind
- 4: return $C(Q)$

Las-Vegas Algorithmus:

RANDOMISED_CLEVERVERSION(P) in $O(n \log n)$

- 1: repeat forever
- 2: wähle $Q \subseteq P$ mit $|Q| = 11$ zufällig und gleichverteilt in $O(n)$ mit Hilfe von Zählern
- 3: bestimme $C(Q)$ in $O(1)$
- 4: if $P \subseteq C^\bullet(Q)$ then in $O(n)$
- 5: return $C(Q)$
- 6: verdoppele alle Punkte von P ausserhalb von $C(Q)$



Den Beweis für die Laufzeit lassen wir aus Zeitgründen weg

Lemma 3.28: Sei P' eine Multimenge mit $|P'| = N$ Punkten

Sei $r \leq N$ und sei $R \in \binom{P'}{r}$ zufällig gleichverteilt (es gilt $|R| = r$ und $R \subseteq P'$)

Sei $X := |P' \setminus C^\bullet(R)|$ die Anzahl Punkte ausserhalb von $C(R)$

Dann ist $\mathbb{E}[X] \leq 3 \cdot \frac{N}{r+1}$

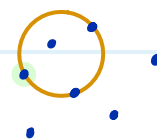
Bsp. $|P'| = 7$

$r = 4$

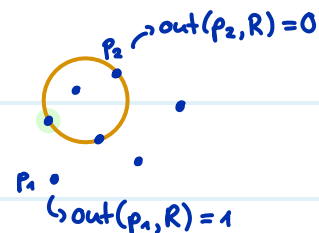
R

$C(R)$

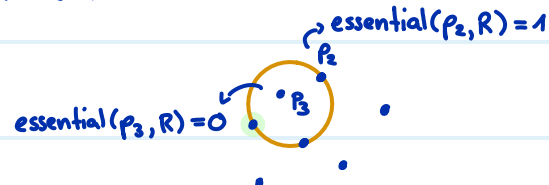
$|X| = 3$



Beweis: wir definieren: $\text{out}(p, R) := \begin{cases} 1 & \text{falls } p \notin C^{\circ}(R) \\ 0 & \text{sonst} \end{cases}$



$\text{essential}(p, Q) := \begin{cases} 1 & \text{falls } C(Q \setminus \{p\}) \neq C(Q) \\ 0 & \text{sonst} \end{cases}$



Für $p \notin R$ gilt: $\text{out}(p, R) = 1 \Leftrightarrow \text{essential}(p, R \cup \{p\}) = 1$

Sei $\Omega = \binom{P'}{r}$ ein Laplace-Raum

$$\mathbb{E}[X] = \sum_{R \in \binom{P'}{r}} \Pr[R] \mathbb{E}[X|R]$$

$$= \sum_{R \in \binom{P'}{r}} \frac{1}{\binom{N}{r}} \sum_{s \in P' \setminus R} \text{out}(s, R)$$

$$= \frac{1}{\binom{N}{r}} \sum_{R \in \binom{P'}{r}} \sum_{s \in P' \setminus R} \text{out}(s, R)$$

$$= \frac{1}{\binom{N}{r}} \sum_{R \in \binom{P'}{r}} \sum_{s \in P' \setminus R} \text{essential}(s, R \cup \{s\})$$

$$= \frac{1}{\binom{N}{r}} \sum_{Q \in \binom{P'}{r+1}} \sum_{p \in Q} \text{essential}(p, Q)$$

$$\leq \frac{1}{\binom{N}{r}} \sum_{Q \in \binom{P'}{r+1}} 3$$

$$= \frac{1}{\binom{N}{r}} \binom{N}{r+1} \cdot 3$$

$$= \frac{(N-r)! \cdot r!}{N!} \cdot \frac{N!}{(N-(r+1))! \cdot (r+1)!} \cdot 3$$

$$= 3 \frac{N-r}{r+1} \leq 3 \frac{N}{r+1}$$

Satz für bedingten Erwartungswert

$$\Pr[R] = \frac{1}{|\Omega|} = \frac{1}{\binom{N}{r}} \quad \text{und} \quad \mathbb{E}[X|R] = \sum_{s \in P' \setminus R} \text{out}(s, R)$$

zählt alle Punkte, die ausserhalb von R liegen

Distributivgesetz

weil $\text{out}(s, R) = \text{essential}(s, R \cup \{s\})$

weil $R \in \binom{P'}{r}$ plus einen weiteren Punkt $s \in P' \setminus R$ gleich $Q \in \binom{P'}{r+1}$ ist.
Wir ersetzen $R \cup \{s\}$ mit Q

es gibt ≤ 3 essential points

$$\left| \binom{P'}{r+1} \right| = \binom{N}{r+1} \quad \text{weil } |P'| = N$$

def. Binomialkoeffizient

kürzen

Flow Aufgaben

Ein Datennetzwerk $N = (V, A, c, s, t)$ besteht aus n Servern und einigen gerichteten Kabelverbindungen dazwischen. Ein Hauptserver s möchte Datenpakete an einen Zielserver t senden. Jedes Kabel kann maximal c Pakete pro Sekunde weiterleiten. Allerdings haben auch die Server selbst eine begrenzte Rechenleistung: Der Server i kann pro Sekunde insgesamt maximal v_i Datenpakete verarbeiten und weiterleiten (egal woher sie kommen und wohin sie gehen).

Modelliere den maximalen Datenfluss von s nach t als ein maxflow Problem.

Wir erstellen ein modifiziertes Netzwerk $N' = (V', A', c', s', t')$ wobei

$$V' = \{i_{in} \mid i \in V\} \cup \{i_{out} \mid i \in V\}$$

wir teilen jeden Knoten in einen "in" und einen "out" Knoten auf

$$A' = \{(i_{out}, j_{in}) \mid (i, j) \in A\} \cup \{(i_{in}, i_{out}) \mid i \in V\}$$

"echte" Kanten zwischen zwei Servern

Verarbeitungskanten

$$c'(x, y) = \begin{cases} v_i & \text{falls } x = i_{in} \text{ und } y = i_{out} \\ c & \text{sonst} \end{cases}$$

→ Verarbeitungskanten

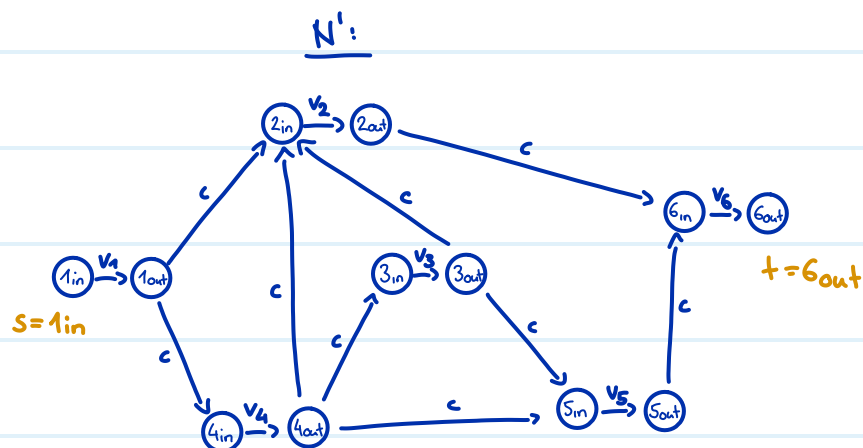
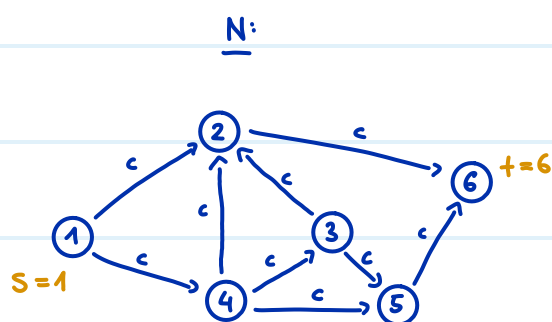
→ "echte" Kanten

$$s' = s_{in}$$

$$t' = t_{out}$$

Der maximale Datenfluss im Datennetzwerk entspricht dem maxflow in N'

Beispiel

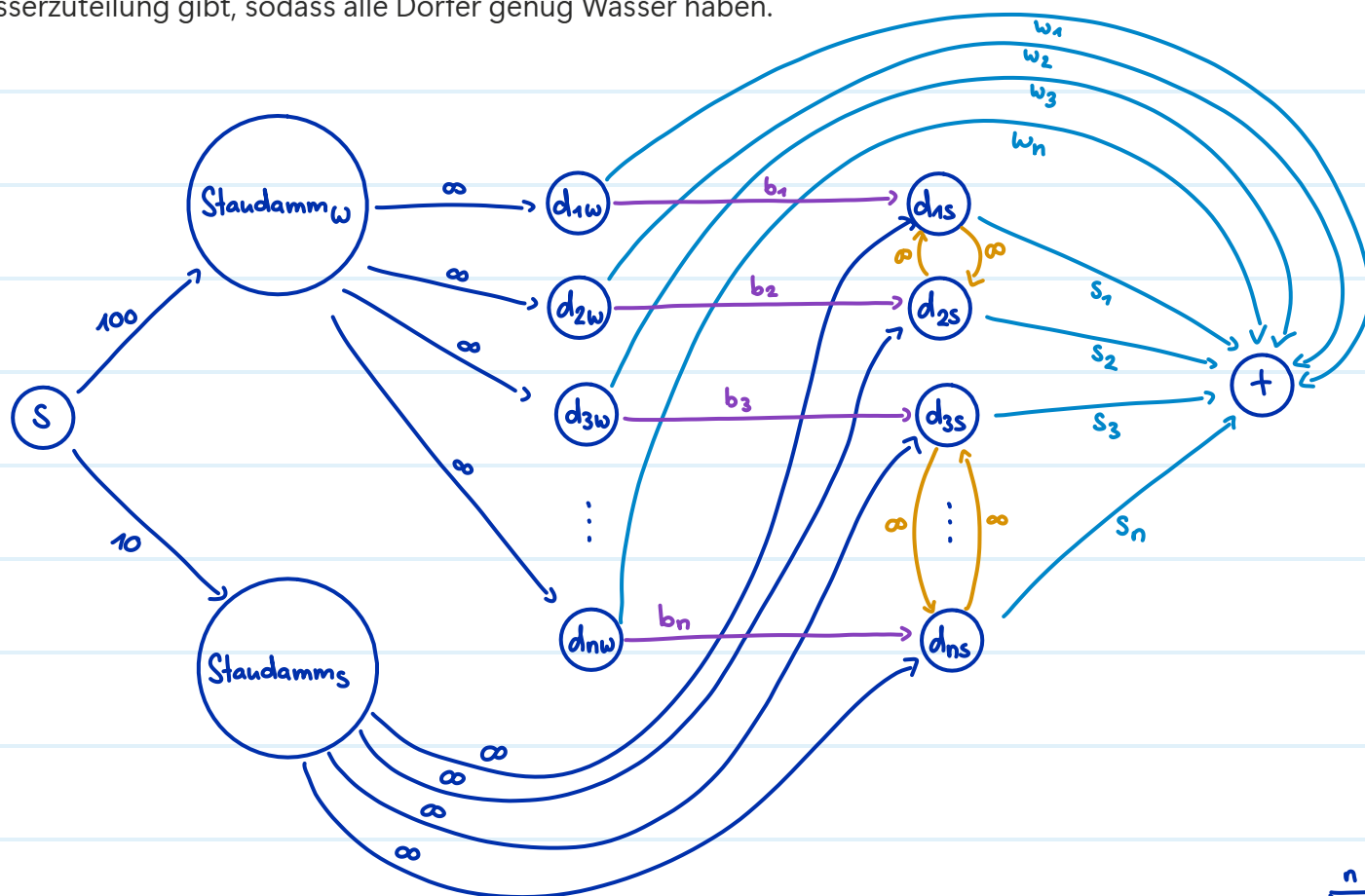


Ein Staudamm im Gebirge versorgt über ein Leitungssystem n Dörfer $d_1, \dots, d_n$ mit Wasser. Es gibt eine separate Leitung ohne Kapazitätsgrenze zu jedem Dorf. Im Winter gibt der Staudamm insgesamt 100 Einheiten Wasser, aber im Sommer nur 10 Einheiten.

Dorf i benötigt im Winter w_i Einheiten Wasser und im Sommer s_i Einheiten Wasser. Dorf i hat auch ein Speicherbecken der Grösse b_i , worin Wasser vom Winter für den Sommer aufbewahrt werden kann.

Einige Dörfer sind miteinander befreundet, d.h. sie können im Sommer gegenseitig Wasserreserven austauschen.

Modelliere dieses Problem als ein maxflow Problem, um festzustellen, ob es eine Wasserzuteilung gibt, sodass alle Dörfer genug Wasser haben.



Zuteilung möglich $\Leftrightarrow \text{maxflow} = \underbrace{\sum_{i=1}^n w_i + s_i}_{\text{Gesamtbedarf}}$

dunkelblaue Kanten = Wasser direkt vom Staudamm

hellblaue Kanten = Wasserverbrauch

violette Kanten = aufbewahrtes Wasser

orange Kanten = Hilfe zwischen befreundeten Dörfern

(Mathematische Definition hier ausgelassen)